

CHƯƠNG 1. INTRODUCTION

1. Một lập trình viên viết 1 chương trình bằng ngôn ngữ Z, khi chạy nhận thấy chương trình bắt đầu chậm nhưng dần trở nên nhanh hơn sau vài lần lặp mà không có biên dịch lại. Phương pháp nào đã được sử dụng?

Đáp án: Just in time Compilation. Vì JIT có chế độ đoạn code nào chạy đi chạy lại hoài thì sẽ biên dịch để tăng tốc độ chạy. Tại sao nhanh dần? Tại vì nó sẽ cần thời gian để phát hiện có sự lặp lại.

2. Một trình dịch lại mất 3 giây để biên dịch mã nguồn thành bytecode, sau đó thực thi với tốc độ 1 giây/mỗi dòng lệnh. Trong khi đó, một trình thông dịch thuần túy mất 2.5 giây để thực thi mỗi dòng lệnh. Số dòng lệnh tối thiểu để trình dịch lại có tổng thời gian nhỏ hơn trình thông dịch thuần túy.

Đáp án: $n = 3$ ($n > 2$)

3. Một chương trình có 2 lựa chọn để thực thi.

Lựa chọn 1: Biên dịch truyền thống mất 7s để biên dịch nhưng sau đó chạy 0.5s/mỗi dòng lệnh.

Lựa chọn 2: Dùng trình dịch tức thời không mất thời gian biên dịch ban đầu nhưng mỗi dòng mất 1.2 giây khi chạy lần đầu và 0.4 giây khi chạy lại.

Với chương trình có n dòng lệnh, trong đó có 40% số dòng được thực thi 2 lần, giá trị của n là bao nhiêu để trình dịch tức thời có tổng thời gian thực thi nhanh hơn biên dịch truyền thống.

Đáp án: $n = 19$ ($n > 18.42$)

4. Công cụ nào có nhiệm vụ kết hợp các file đối tượng lại với nhau và tạo ra một tệp thực thi.

Đáp án: Linker

5. Điền vào chỗ trống:

..... đảm bảo rằng tất cả các chương trình và thư viện cần thiết được nạp vào bộ nhớ trước khi thực thi.

Đáp án: Loader

CHƯƠNG 2. PHÂN TÍCH TỪ VỰNG (CÓ HARMONY)

1. Cho các lỗi sau:

- 1 biến chưa được khai báo nhưng được sử dụng (**Lỗi name - Static Checking**)
- 1 từ khóa được sử dụng làm tên biến (**Lỗi lexer**)
- Dấu @ xuất hiện trong mã nguồn của ngôn ngữ C (**Lỗi Lexer**)
- Một dấu ngoặc bị đóng thiếu (**Lỗi Parser**)
- Một số nguyên có dấu chấm thập phân (**Lỗi Lexer**)

Có bao nhiêu lỗi trong danh sách trên mà bộ phân tích từ vựng có thể phát hiện?

Đáp án: 3 lỗi

2. Cho đoạn mã sau được viết bằng C++:

```
int lvar = 10;  
float num = 12.;  
char ch = '$';
```

Có bao nhiêu lỗi mà bộ phân tích từ vựng có thể phát hiện lỗi trong đoạn mã trên?

Đáp án: 2 lỗi - tên biến bắt đầu bằng số 1, số thực không có phần thập phân

3. Trong một tập luật của ANTLR4, ta có quy tắc sau:

```
STRING: '"' .*? '"';
```

Quy tắc này cho phép nhận vào mọi chuỗi ở giữa 2 dấu nháy kép. Ta muốn thay đổi action để mọi chuỗi “hello” đều được thay bằng chuỗi “STRING CONSTANT” khi token được xử lý. Hãy mô tả cách thực hiện điều này.

Đáp án:

```
if self.text == "\"hello\""
```



```
if self.text[1:-1] == "hello"
```

 (2 cách)

4. Một bộ phân tích từ vựng sử dụng quy tắc sau:

```
MULTILINE_COMMENT: '/*' .*? '*/' -> skip;  
SINGLELINE_COMMENT: '//' ~[\r\n]* -> skip;
```

Tuy nhiên, khi biên dịch một tệp tin mã nguồn chứa nhiều dòng chú thích, chương trình đôi khi gặp lỗi hoặc hoạt động không như mong muốn. Hãy xác định các tình huống lỗi có thể xảy ra với quy tắc trên.

Đáp án:

- **Multiline comment: Không hỗ trợ Nested Comment**
- **Single Line comment: Tùy vào hệ điều hành mà \r\n sẽ khác nhau**

5. Cho biểu thức chính quy: $(0|1)^*101$. Hãy thiết kế 1 DFA để nhận diện tập hợp chuỗi thỏa mãn biểu thức này. Trong đó, số trạng thái tối thiểu cần có của DFA là bao nhiêu?

Đáp án: 4

6. Một chuỗi nhị phân được gọi là đặc biệt nếu nó chứa đúng 2 lần sự xuất hiện của dãy 101 và không chứa dãy 110 ở bất kỳ vị trí nào.

- Hãy viết biểu thức chính quy mô tả cho tập hợp chuỗi đặc biệt này
- Hãy xây dựng DFA tối thiểu để nhận diện chuỗi đặc biệt trên

CHƯƠNG 3. PHÂN TÍCH CÚ PHÁP (LEXER ANALYSIS)

1. Cho các lỗi sau đây:

- 1 biến được sử dụng nhưng chưa khai báo (**Lỗi ngữ nghĩa**)
- 1 hàm được gọi với số lượng tham số bị sai (**Lỗi ngữ nghĩa**)
- 1 phép toán chia cho 1 số 0 (**Lỗi thực thi**)
- Một vòng lặp vô hạn không có điều kiện dừng (**Lỗi thực thi**)
- Một cú pháp sai do quên dấu ; khi lập trình (**Lỗi cú pháp**)

Có bao nhiêu lỗi trong danh sách trên mà bộ phân tích cú pháp có thể phát hiện?

2. Trong một ngôn ngữ lập trình, một toán tử có thể được định nghĩa theo văn phạm sau:

$E \rightarrow E + E \mid E * E \mid (E) \mid id$

***Note: id là ký hiệu kết thúc**

a. Giải thích tại sao văn phạm này bị nhập nhằng, vì sao? (**Lấy ví dụ sinh ra được 2 cây**)

b. Hãy đề xuất cách chỉnh sửa văn phạm này để đảm bảo độ ưu tiên của toán tử

3. Cho văn phạm phi ngữ cảnh:

$S \rightarrow aSbS \mid \epsilon$

a. Chứng minh rằng, văn phạm này sinh ra tất cả các chuỗi có số lượng ký tự a bằng số lượng ký tự b , nhưng không nhất thiết phải ở dạng $a^n b^n$

b. Chứng minh rằng văn phạm trên không nhập nhằng bằng cách chỉ ra mọi chuỗi hợp lệ có duy nhất một cây phân tích cú pháp

4. Cho hai văn phạm phi ngữ cảnh:

$S \rightarrow aSbc \mid abc$

$S \rightarrow abSc \mid abc$

Hai văn phạm này có cùng sinh ra một ngôn ngữ hay không? Giải thích

CHƯƠNG 4. OOP

1. Trong OOP, ràng buộc động (dynamic binding) và tính đa hình có mối quan hệ mật thiết với nhau.

a. Giải thích vai trò của ràng buộc động trong việc hỗ trợ tính đa hình

b. Nếu ngôn ngữ lập trình không có hỗ trợ ràng buộc động thì cách duy nhất để hỗ trợ tính đa hình là gì? Nêu ví dụ về một ngôn ngữ cụ thể

Đáp án:

b. Sử dụng Template

2. Một số ngôn ngữ như C++ cho phép ràng buộc tĩnh và động, trong khi các ngôn ngữ khác như Python thực hiện ràng buộc động hoàn toàn. Hãy so sánh ràng buộc động trong Python với cách hoạt động của từ khóa **virtual** trong C++

b. Điều gì sẽ xảy ra nếu một phương thức **virtual** trong C++ lại không được khai báo bằng **virtual**. Nó có ảnh hưởng gì tới tính đa hình hay không?

3. C++ có các mức độ truy cập là **public**, **protected** hoặc **private** trong khi Python lại không có từ khóa kiểm soát mà lại thực hiện bằng quy ước.

a. Tại sao Python lại không bắt buộc các mức độ truy cập như trong C++

b. Hãy viết 2 đoạn mã trong C++ và Python để minh họa các thuộc tính private từ bên ngoài của lớp. Python có thực sự ngăn chặn được như C++ hay không?

4. Nội dung về MRO (chắc chắn có thi). Ví dụ

Xét một đoạn mã Python sau:

```
1 class A:
2     def __init__(self):
3         print("A", end="")
4
5 class B(A):
6     def __init__(self):
7         print("B", end="")
8         super().__init__()
9
10 class C(A):
11     def __init__(self):
12         print("C", end="")
13         super().__init__()
14
15 class D(B, C):
16     def __init__(self):
17         print("D", end="")
18         super().__init__()
19
20 D()
```

- Kết quả của chương trình trên là gì?
- Giải thích thứ tự gọi phương thức init trong các lớp theo MRO
- Nếu thay dòng super trong class B C thành A.__init__() thì kết quả có thay đổi không? Giải thích

5. Functional Programming (Python)

- immutable, mutable khác nhau như thế nào
- currying function
- decorator